

Introdução às Linguagens Formais

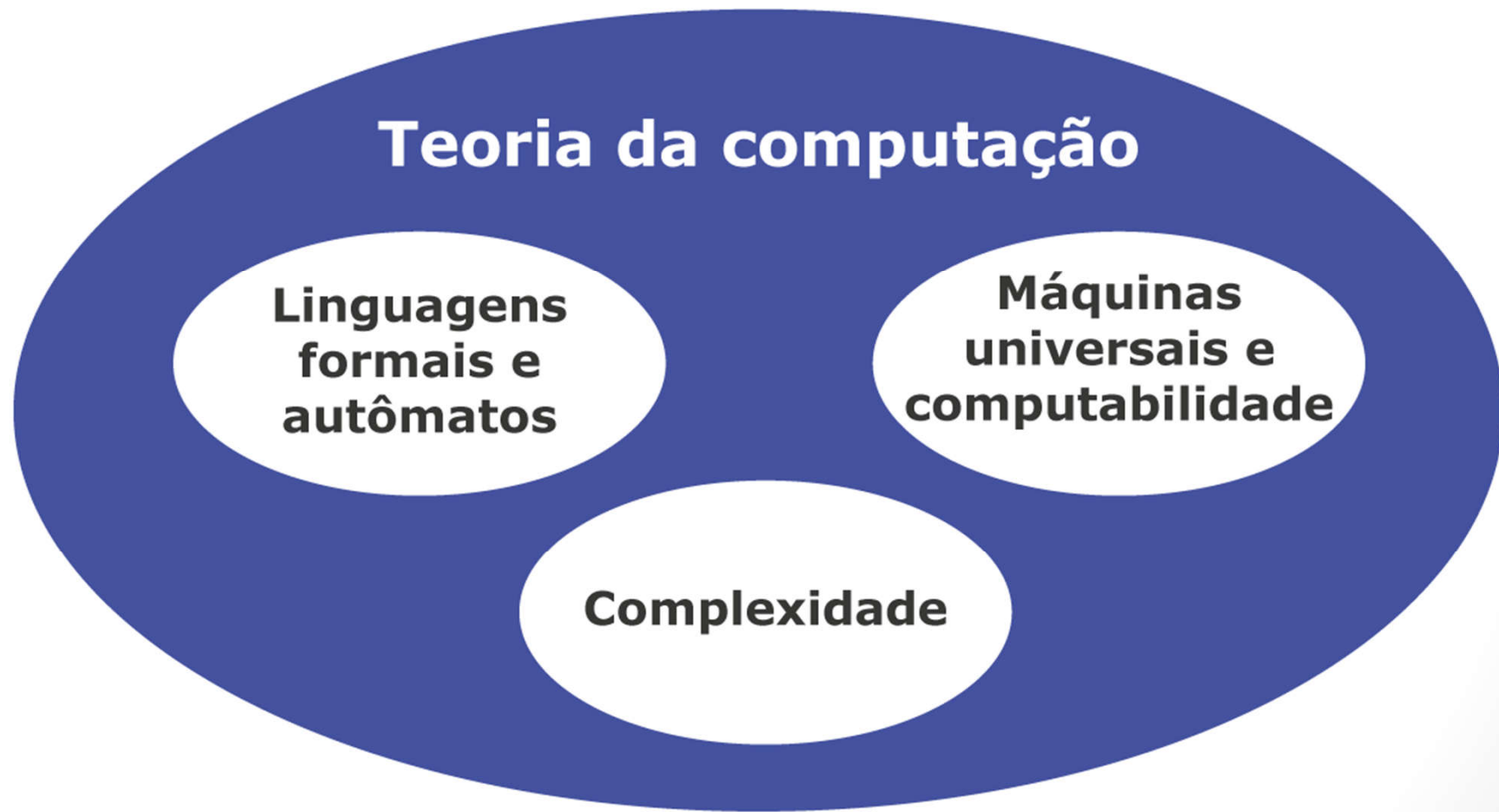
Professor Diógenes Furlan

Teoria da Computação e Linguagens Formais

UTP

Aula 01

Principais Áreas



Principais Áreas

- **Teoria dos Autômatos (Linguagens Formais)**
 - lida com as definições e propriedades de modelos matemáticos de computação.
- **Teoria da Complexidade**
 - classificar os problemas como “fáceis” ou “difíceis”.
- **Teoria da Computabilidade**
 - classificar os problemas como “solúveis” ou “insolúveis”.

Sumário

- Noções de Complexidade
- Teoria da Computação
- Linguagens Formais
- Conceitos iniciais de Linguagens Formais

COMPLEXIDADE

Complexidade de Tempo

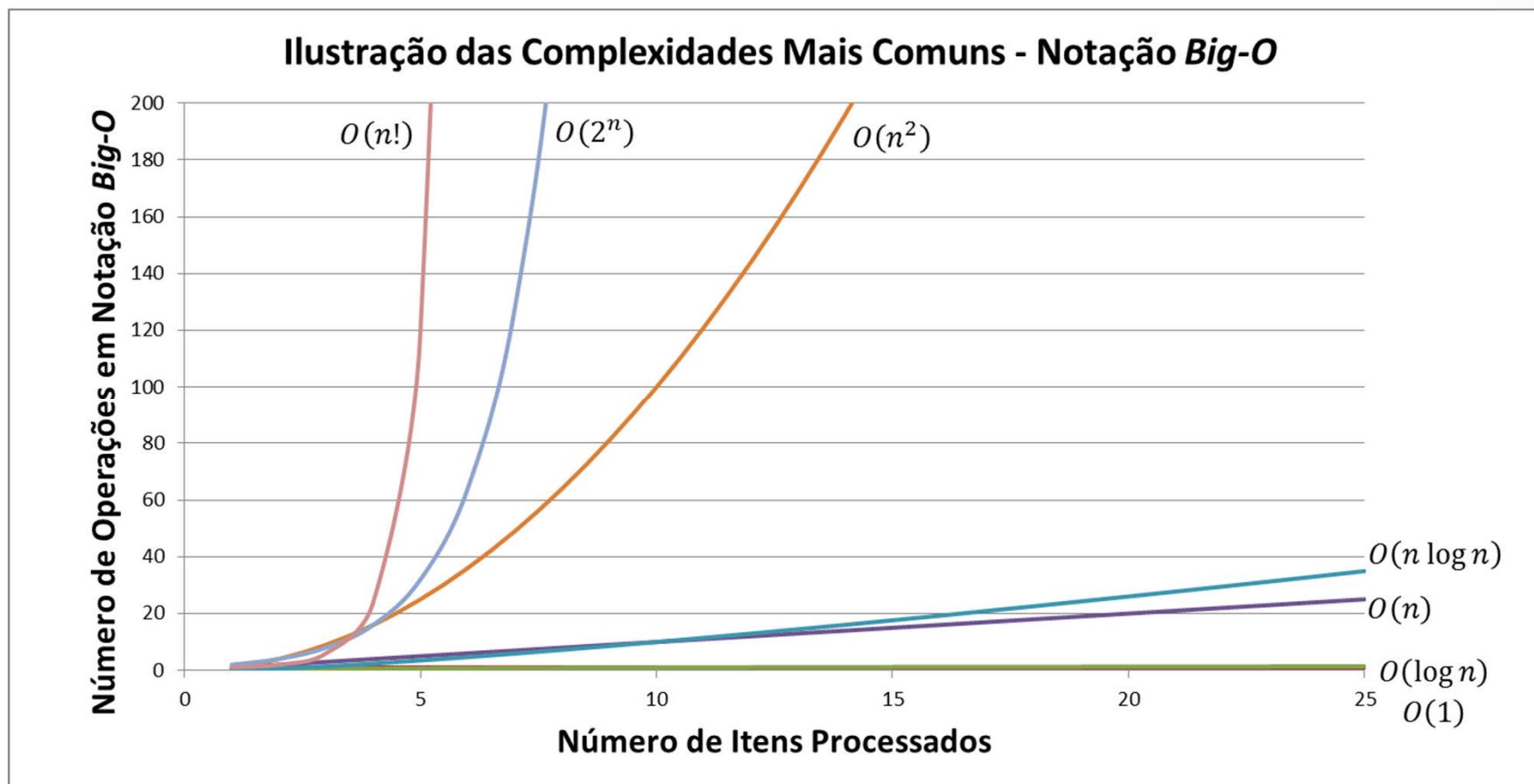
- Problemas Fáceis

- Problema de Ordenação: arranjar uma lista de números em ordem ascendente
- 1.000.000 números → segundos

- Problemas Difíceis

- Problema do Escalonamento: encontrar um escalonamento de aulas para a universidade inteira, que satisfaça algumas restrições razoáveis.
 - como 2 aulas não podem ter lugar na mesma sala ao mesmo tempo.
- 1000 aulas → séculos

Relação entre as Categorias de Problemas



<http://i.stack.imgur.com/R6E94.png>

O que é mais importante??!!??

MÁQUINA X ALGORITMO

Algoritmos e Tecnologia

O mundo real: computadores têm velocidade de processamento e memória limitadas.

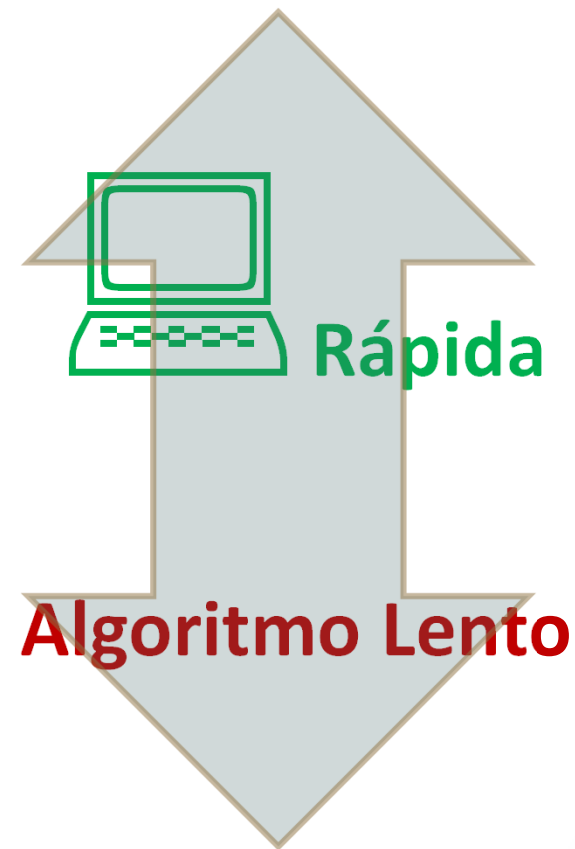
Neste caso faz muita diferença ter um bom algoritmo.

Algoritmos e Tecnologia

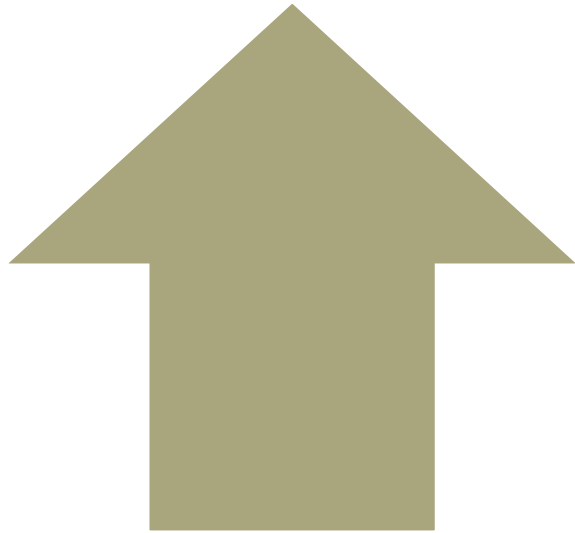
Exemplo: **ordenação de um vetor de n elementos**

- Suponha que os computadores A e B executam 1G e 10M instruções por segundo, respectivamente.
 - Ou seja, A é 100 vezes mais rápido que B.
- **Algoritmo Lento**: implementado na máquina A por um programador mediano em linguagem de alto nível dispondo de um compilador “meia-boca”.
 - Executa $2n^2$ instruções.
- **Algoritmo Rápido**: implementado em B por um excelente programador em linguagem de máquina (ultra rápida).
 - Executa $50n \cdot \log n$ instruções.

Relacionamiento

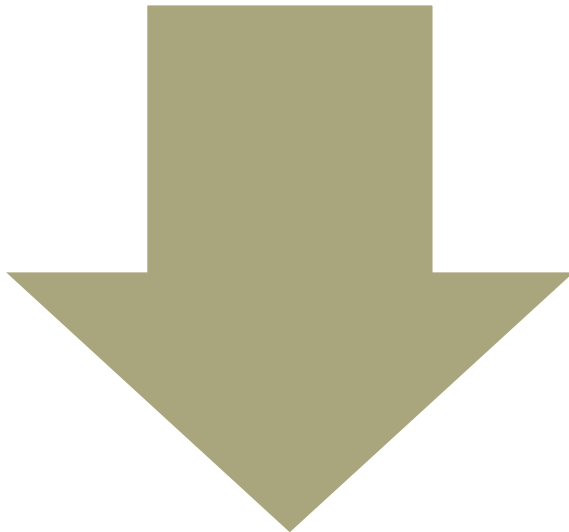


Algoritmos e Tecnologia



1GHz = 1×10^9
instruções/segundo

Bolha: $2.n^2$



10MHz = 1×10^7
instruções/segundo

Quicksort: $50.n \log n$

Algoritmos e Tecnologia

- O que acontece quando ordenamos um vetor de **um mil elementos**?
- Qual par (máquina/algoritmo) é mais rápido?

- Algoritmo L na máquina A:

$$\frac{2 \cdot (10^3)^2 \text{ instruções}}{10^9 \text{ instruções/segundo}} \approx 0,002 \text{ segundos}$$

HW bom

- Algoritmo R na máquina B:

$$\frac{50 \cdot 10^3 \cdot \log 10^3 \text{ instruções}}{10^7 \text{ instruções/segundo}} \approx 0,015 \text{ segundos}$$

SW bom

- Ou seja, B(R) foi 7,5 VEZES mais lento do que A(L)!

Algoritmos e Tecnologia

- O que acontece quando ordenamos um vetor de **um milhão de elementos**?
- Qual par (máquina/algoritmo) é mais rápido?

- Algoritmo L na máquina A:

$$\frac{2 \cdot (10^6)^2 \text{ instruções}}{10^9 \text{ instruções/segundo}} \approx 2.000 \text{ segundos}$$

HW bom

- Algoritmo R na máquina B:

$$\frac{50 \cdot 10^6 \cdot \log 10^6 \text{ instruções}}{10^7 \text{ instruções/segundo}} \approx 30 \text{ segundos}$$

SW bom

- Ou seja, B(R) foi 66 VEZES mais rápido do que A(L)!

Algoritmos e Tecnologia

- O que acontece quando ordenamos um vetor de **um bilhão de elementos**?
- Qual par (máquina/algoritmo) é mais rápido?

- Algoritmo L na máquina A:

$$\frac{2 \cdot (10^9)^2 \text{ instruções}}{10^9 \text{ instruções/segundo}} \approx 2.000.000.000 \text{ segundos}$$

63 anos e 153 dias

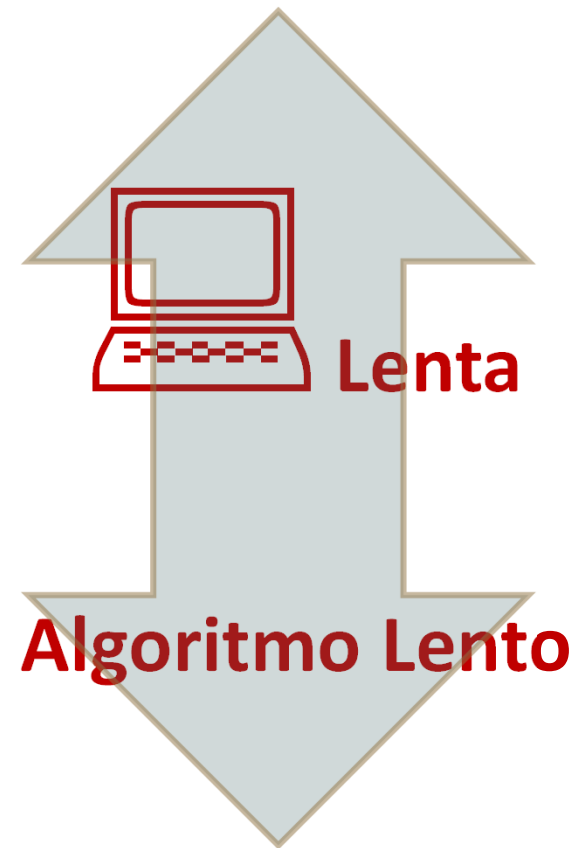
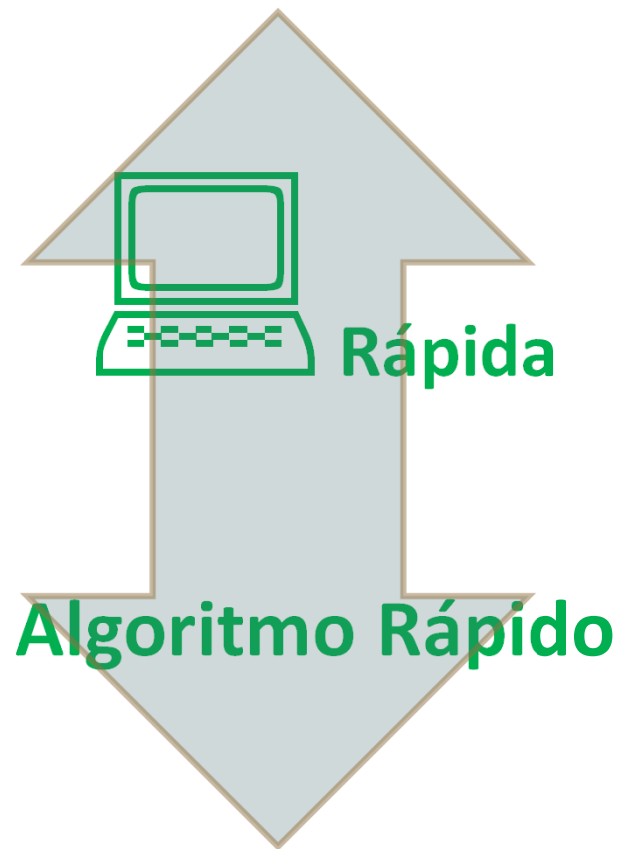
- Algoritmo R na máquina B:

$$\frac{50 \cdot 10^9 \cdot \log 10^9 \text{ instruções}}{10^7 \text{ instruções/segundo}} \approx 45.000 \text{ segundos}$$

12 horas e 30 minutos

- Ou seja, B(R) foi 44.444 VEZES mais rápido do que A(L)!

Relacionamiento



Invertendo tudo

- O que acontece quando ordenamos um vetor de **um bilhão de elementos**?
- Qual par (máquina/ algoritmo) é mais rápido?

- Algoritmo L na máquina B:

$$\frac{2 \cdot (10^9)^2 \text{ instruções}}{10^7 \text{ instruções/segundo}} \approx 200.000.000.000 \text{ segundos}$$

6 milênios e 342 anos

- Algoritmo R na máquina A:

$$\frac{50 \cdot 10^9 \cdot \log 10^9 \text{ instruções}}{10^9 \text{ instruções/segundo}} \approx 450 \text{ segundos}$$

7 minutos e 30 segundos

- Ou seja, A(R) foi 444.444.444 VEZES mais rápido do que B(L)!

Tabela de segundos

Período	Número de Segundos
segundo	1
minuto	60
hora	3.600
dia	
ano	
milênio	

Tabela de segundos

Período	Número de Segundos
segundo	1
minuto	60
hora	3.600
dia	86.400
ano	
milênio	

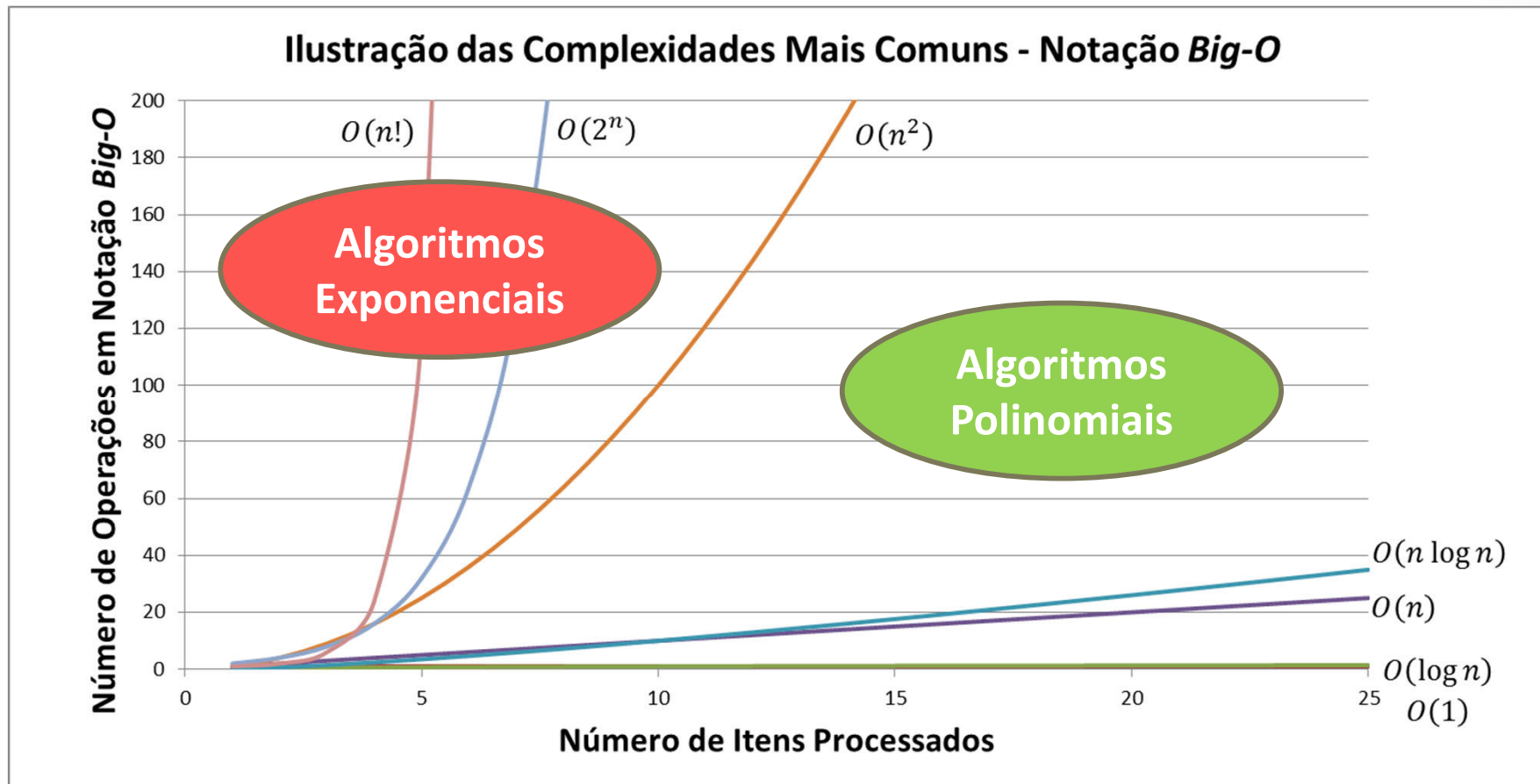
Tabela de segundos

Período	Número de Segundos
segundo	1
minuto	60
hora	3.600
dia	86.400
ano	31.536.000
milênio	

Tabela de segundos

Período	Número de Segundos
segundo	1
minuto	60
hora	3.600
dia	86.400
ano	31.536.000
milênio	31.536.000.000

Relação entre as Categorias de Problemas



<http://i.stack.imgur.com/R6E94.png>

TEORIA DA COMPUTAÇÃO

Computabilidade

- Seu objetivo é investigar a existência ou ausência de algoritmos que solucionem determinada classe de problemas.
 - Evitar a pesquisa de soluções inexistentes
- Exemplo: em 1901, o matemático David Hilbert formulou uma lista de problemas a serem resolvidos pelas futuras gerações de matemáticos (23 problemas de Hilbert).
 - **10º problema:** Existe ou não um algoritmo que determine se uma equação polinomial qualquer, com coeficientes inteiros, possui solução nos inteiros?
- Somente em 1970, Matijasevic provou que este problema é “sem solução”.

Categorias básicas de problemas

- **Problemas de decisão:** problemas com respostas binárias, do tipo sim ou não.

DECIDIBILIDADE

- **Problemas de busca:** problemas onde se há uma solução, pelo menos uma delas é devolvida; se não há, retorna negativo com uma mensagem apropriada.
- **Problemas de otimização:** problemas onde se deve encontrar a melhor solução de todas as soluções viáveis.

Categorías básicas de problemas



Créditos: ABB Photo/Shutterstock

Relação entre Problemas



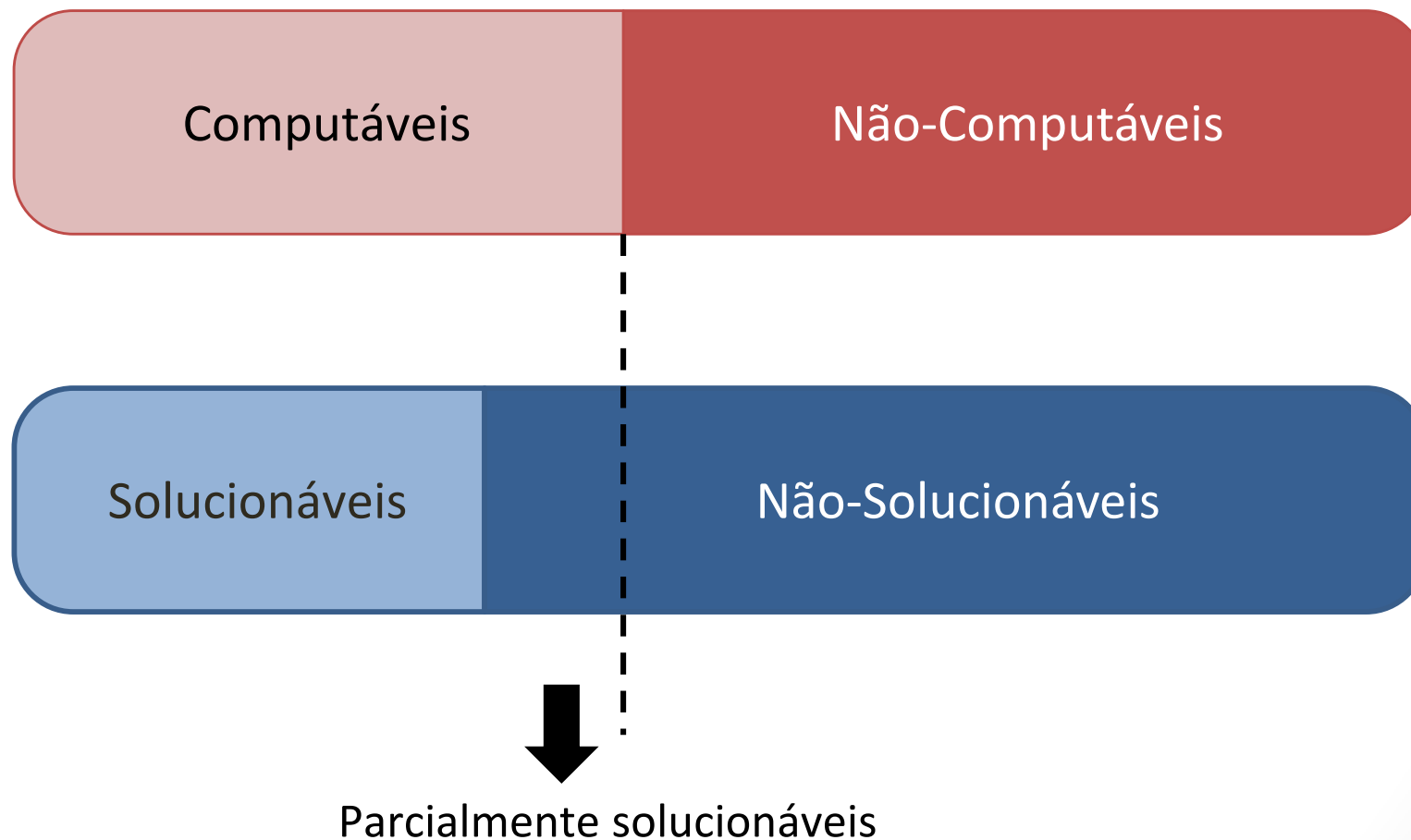
Classificações de Problemas

- **Problema Solucionável:** é aquele para o qual existe um algoritmo que solucione o problema e sempre pára, para toda entrada, com uma resposta afirmativa (ACEITA) ou negativa (REJEITA).
- **Problema Computável:** é aquele para o qual existe uma Máquina Universal capaz de resolvê-lo.
 - $\text{sizeof}(\text{problemas computáveis}) \ll \text{sizeof}(\text{problemas não computáveis})$

Computabilidade

- Problemas Solúveis
 - Problema de Ordenação: arranjar uma lista de números em ordem ascendente
- Problemas Insolúveis
 - Problema do Enunciado: determinar se um enunciado matemático é verdadeiro ou falso.
 - reside estritamente dentro do domínio da matemática
 - é o ganha pão dos matemáticos
 - eles queriam saber se existia um algoritmo capaz de simular o que um matemático faz, pois assim os matemáticos se tornariam “dispensáveis”.

Classificações de Problemas



Hipótese de Church

- **Maquina de Turing:** é uma tentativa de definir formalmente a noção intuitiva de algoritmo
- Diversos trabalhos “independentes” resultaram em conceitos equivalentes ao de Turing:
 - Máquina de Post
 - Máquina Norma
 - Autômato de Pilhas
 - Funções Recursivas (Kleene, 1936)

Hipótese de Church

“A capacidade de computação representada pela Máquina de Turing é o limite máximo que pode ser atingido por qualquer dispositivo de computação.”

LINGUAGENS FORMAIS

Como representar uma linguagem?

- **Linguagem Finita:** É uma Linguagem que pode ser representada por enumeração.
 - A linguagem definida como sendo o conjunto dos inteiros positivos pares maiores que 0 e menores que 20.
 - $L = \{2, 4, 6, 8, 10, 12, 14, 16, 18\}$.
- **Linguagem Infinita:** Neste caso, na impossibilidade de usarmos enumeração, precisamos encontrar uma representação finita para estas linguagens.
 - A linguagem definida como sendo o conjunto dos inteiros pares.
 - $V = \{2, 4, 6, 8, 10, \dots\}$
 - É intuitiva, mas não é finita e nem precisa.
- Conjuntos infinitos podem ainda ser classificados em:
 - contáveis (quando tem correspondência com os números naturais)
 - incontáveis (quando tem correspondência com os números reais).

Linguagens Infinitas

- LI contável
 - Naturais Pares

Contagem	1º	2º	3º	4º	5º	6º	7º	8º	...
Elemento	2	4	6	8	10	12	14	16	...

- LI incontável
 - Reais entre 0 e 1

	1º	2º	...	Qtd
	0.1	0.2	...	10
	0.01	0.02	...	100
	0.001	0.002	...	1000
	0.0001	0.0002	...	10000

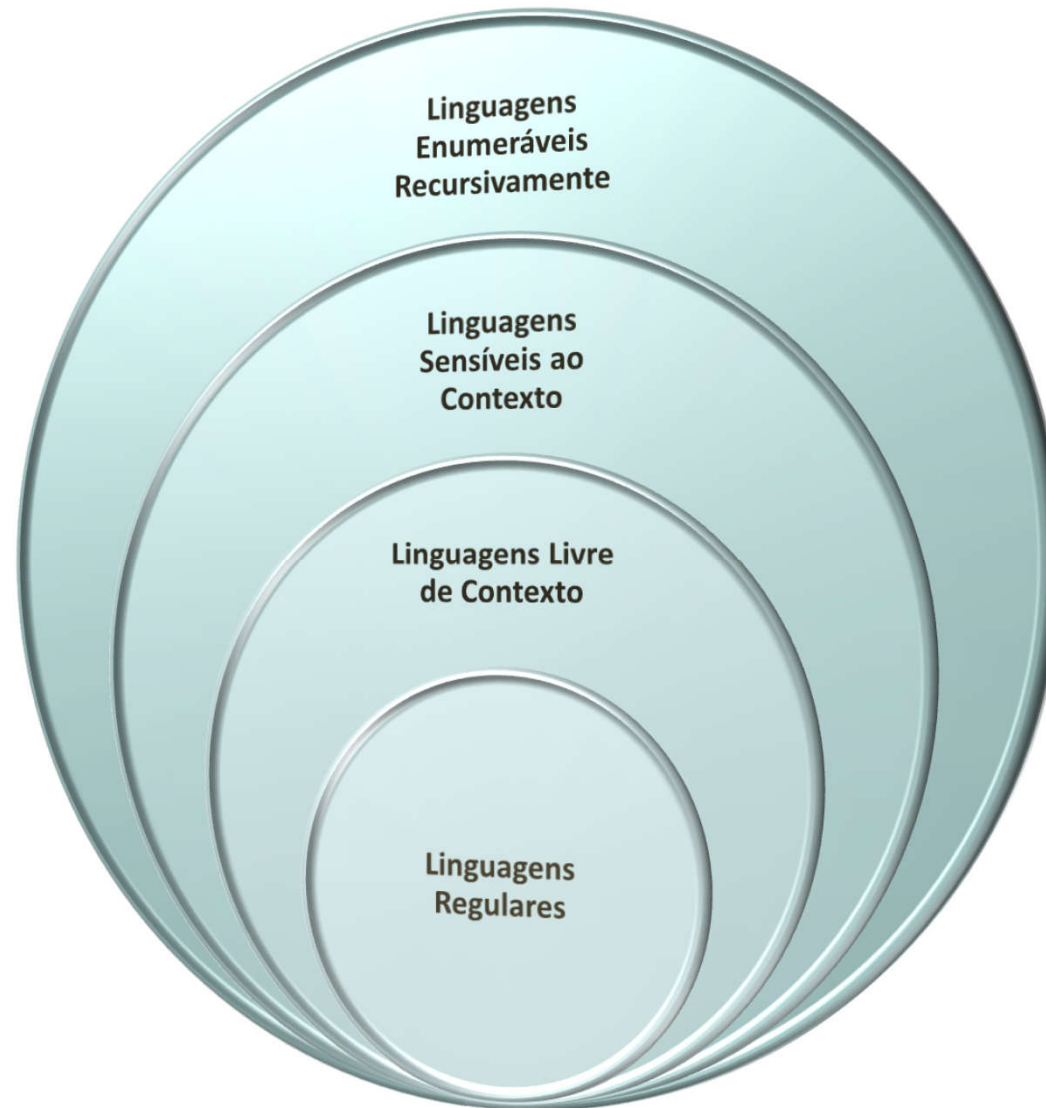
Linguagem Formal

- Uma **Linguagem Formal** é uma linguagem que podem ser representadas de maneira finita e precisa através de sistemas com sustentação matemática
 - dispositivos formais ou modelos matemáticos.

Representações Finitas

- **Descritores** – são formalismos que representam os elementos de uma linguagem, descrevendo cada um deles usando uma notação matemática.
 - calculo proposicional, calculo de predicados, expressões regulares, ...
- **Reconhecedores** – São dispositivos formais que nos permitem verificar se uma determinada sentença pertence ou não a uma determinada linguagem.
 - autômatos finitos, autômatos de pilha e máquinas de Turing
- **Geradores** – São dispositivos formais dotados de mecanismos que permitem a geração sistemática das sentenças de uma linguagem.
 - gramáticas de CHOMSKY.

Hierarquia de Chomsky



Linguagens Regulares

- Classe de linguagens mais simples.
- Não requer memória para fazer o reconhecimento.
- Complexidade:
 - Pode ser resolvida por uma máquina de Turing determinística sem usar espaço adicional.
- Formalismos:
 - Denotacional ou Descritor: Expressão Regular
 - Operacional ou Reconhecedor: Autômato Finito
 - Axiomático ou Gerador: Gramática Regular

Linguagens Livre de Contexto

- Formalismos:
 - Operacional ou Reconhecedor: Autômato de Pilha
 - Axiomático ou Gerador: Gramática Livre de Contexto
- Exemplos
 - Linguagens de Programação que requerem o **balanceamento de parênteses**
 - Expressões aritméticas

Linguagens Sensíveis ao Contexto

- Formalismos:
 - Operacional ou Reconhecedor: Autômato Linearmente Limitado
 - Axiomático ou Gerador: Gramática Sensível ao Contexto
- Complexidade:
 - Pode ser resolvida por uma máquina de Turing não determinística usando espaço linear.

Linguagens Enumeráveis Recursivamente

- Formalismos:
 - Operacional ou Reconhecedor:
 - Máquina de Turing
 - Máquina de Post
 - Máquina de 2 Pilhas
 - Máquina Norma
 - Axiomático ou Gerador: Gramática Irrestrita

CONCEITOS INICIAIS DE LFA

Alfabeto

- Um **Símbolo** $\langle\langle a,b,c,\dots \rangle\rangle$ é um elemento de um conjunto, representado por algarismos (letras, caracteres, números, ...).
- Um **Alfabeto** (ou vocabulário) $\langle\langle \Sigma \rangle\rangle$ é um conjunto finito, e não vazio, de símbolos.
 - BINARIO = $\{0,1\}$
 - LETRAS = $\{a,b,\dots,z\} = [abcdefghijklmnopqrstuvwxyz]$
 - TECLADO = $[ab\dots zAB\dots Z01\dots 9+-*/'!"@#\$%\textasciitilde\&()[]\{\}\textasciigrave\^{\sim}_=<>,:;?\backslash |]$
 - LING_C = $[ab\dots zAB\dots Z01\dots 9+-*/'!"\#\%&()[]\{\}_=<>,:;?\backslash |]$
 - $\Sigma_5 = \text{ASCII}$
 - $\Sigma_6 = \text{EBCDIC}$
 - $\Sigma_7 = \text{UTF-8}$

LING_C $\subset$ ASCII

Palavra

- Uma **Palavra** $\langle\langle x,y,w,\dots \rangle\rangle$ é uma sequência (ou cadeia) finita de símbolos do alfabeto justapostos por concatenação.
- Sinônimos: sentença, string, cadeia
 - Seja $\Sigma = \{a,b\}$
 - então $a, b, aa, ab, aba, \varepsilon, \dots$ são palavras.
- A **Palavra vazia** $\langle\langle \varepsilon \rangle\rangle$ é uma sentença constituída de nenhum símbolo; isto é, uma palavra de comprimento igual a 0 (zero). Representamos a sentença vazia por ε (épsilon).

Palavras

- Para a linguagem C, temos as seguintes “palavras”/“programas”:
 - `void main() {}`
 - `void main() { printf(""); };`
 - `void main() { int a; }`
 - `void main() { int a; printf(""); };`
 - `void main() { int a,b,c; }`
 - `int a; void main() { printf(""); };`
 - `void main() { int x; x=10; }`
 - ...

Linguagem

- Uma Linguagem $\langle\langle L \rangle\rangle$, sobre um alfabeto Σ , é um subconjunto de Σ^* ; isto é,

$$L \subseteq \Sigma^*$$

- ou seja, é um conjunto de palavras ou de sentenças.
- Seja $\Sigma = \{a, b\}$
 - $L_1 = \emptyset$
 - $L_2 = \{\epsilon\}$
 - $L_3 = \{w \mid w \text{ é palíndromo}\} = \underline{\hspace{10cm}}$
 - $L_4 = \Sigma^*$

EXERCÍCIOS

Exercícios

1. Quais dos seguintes conjuntos são alfabetos?
 - a) números racionais
 - b) letras gregas
 - c) algarismos arábicos
 - d) $2^{\{a,b,c\}}$
 - e) $\{a,b,c\}^3$
 - f) números primos

2. Mostre os prefixos e os sufixos de 001122. Se preferir gere um algoritmo para fazer isso.

Exercícios

3. Definimos Σ^k como o conjunto de strings de comprimento k sobre o alfabeto Σ .

- Seja $\Sigma = \{0,1,2\}$
- Descreva:
 - $\Sigma^2 =$ _____
 - $\Sigma^3 =$ _____
 - $\Sigma^4 =$ _____
 - $\Sigma^0 =$ _____
 - $\Sigma^* =$ _____
 - $\Sigma^+ =$ _____

Exercícios

4. Dê alguns exemplos de palavras que pertencem (e que não pertencem) às seguintes linguagens, sendo $\Sigma = \{a, b\}$. Por exemplo:

$$L = \{w \mid w \in \Sigma\Sigma\}$$

válido: ab

inválido: aba

a) $L = \{w \mid w \in \Sigma^3\}$

b) $L = \{w \mid \text{para algum } u \in \Sigma\Sigma, w = uau\}$

c) $L = \{w \mid \text{para algum } u \in \Sigma\Sigma, w = uu^R u\}$ $u^R \rightarrow$ reverso de u

d) $L = \{w \mid ww = www\}$

e) $L = \{w \mid \text{para algum } u, v \in \Sigma^*, uvw = wvu\}$

f) $L = \{w \mid \text{para algum } u \in \Sigma^*, www = uu\}$

Exercícios

5. Produzir um algoritmo (ou código Python/C++) que mostre todas as palavras de 4 letras sobre o alfabeto $\Sigma = \{a,b,c,d,e\}$.
6. Existe como produzir um algoritmo que gere Σ^* ?